

Error Detection & Correction

1. Why This Matters

Every transmission medium is imperfect — electrical noise, attenuation, interference, and collisions can flip bits in transit. The Data Link layer (and, at a higher level, the Transport layer via checksums) needs a way to tell whether received data matches what was sent, and in some cases, to actually fix the corruption without needing a retransmission.

Types of errors

- **Single-bit error** — exactly one bit in the data unit is flipped.
- **Burst error** — two or more *consecutive* bits are flipped (common on real links, since noise typically affects a contiguous stretch of signal, not an isolated bit).

```
Original:  1 0 1 1 0 0 1 0
Single-bit: 1 0 1 0 0 0 1 0   (bit 4 flipped)
Burst:      1 0 0 0 1 0 1 0   (bits 3-5 flipped, a contiguous run)
```

2. Single Parity Check

Add **one** extra bit so the total number of 1s in the unit (data + parity bit) matches a chosen convention:

- **Even parity** — total number of 1s must be even.
- **Odd parity** — total number of 1s must be odd.

Worked example (even parity): sending the word "world" in ASCII, one character at a time — each 7-bit ASCII character gets an 8th parity bit appended so the total 1-count is even.

```
'w' = 1110111 (has six 1s → already even → parity bit = 0) → 11101110
```

At the receiver: count the 1s in each received character.

- If corrupted: **11111110** — seven 1s (odd) → the receiver knows this violates even parity → **error detected**, discard and request retransmission.

Limitation: single parity can only reliably detect an **odd number of bit flips**. If exactly 2 bits flip (an even number), the total count of 1s is unchanged, and the error slips through undetected entirely.

3. Two-Dimensional Parity Check

Improves on single parity by arranging data into a **grid (rows × columns)** and computing a parity bit for **every row and every column**.

```
Data blocks: 10101001 11100111 11011101 00111001
```

```
1 0 1 0 1 0 0 1 → row parity
1 1 1 0 0 1 1 1 → row parity
1 1 0 1 1 1 0 1 → row parity
0 0 1 1 1 0 0 1 → row parity
↓ column parities
```

Why this is stronger: a single-bit error now fails **both** its row's parity check **and** its column's parity check simultaneously — this doesn't just detect the error, it actually **pinpoints its exact location** (the intersection of the failing row and failing column), though 2D parity by itself is still fundamentally a *detection* scheme, not a *correction* scheme (you know *where* the bad bit is, but not what value it should have been — though for a single bit, flipping it back is the obvious fix, which blurs into simple correction in that specific 1-bit case).

Limitation: certain *specific patterns* of 4 simultaneously corrupted bits (forming a rectangle at the intersections of exactly 2 rows and 2 columns) can cancel out and go completely undetected — each affected row and column still shows an even number of flipped bits (2), invisible to a parity check that only counts total 1s per row/column.

4. Checksum

Used prominently at the **Transport layer** (both UDP and TCP), not just the Data Link layer.

Algorithm (sender):

1. Divide the data into **k** sections of **n** bits each.
2. Add all sections together using **one's complement addition** (any carry-out from the leftmost bit wraps around and is added back in).
3. **Complement** (flip every bit of) the final sum — this is the **checksum**.
4. Send the original data **plus** the checksum.

Worked example: data = 10101001 00111001

```
10101001
+ 00111001
-----
11100010 ← sum
```

Complement the sum → 00011101 ← **checksum**

Transmitted: 10101001 00111001 00011101

At the receiver: add up **all** received sections (data + checksum) using the same one's complement addition.

```

 10101001
+ 00111001
+ 00011101
-----
 11111111 ← sum

```

Complement it → **00000000** — **all zeros means no error detected**. (If any bit had been corrupted in transit, the complemented sum would contain at least one 1.)

Interview soundbite: "The checksum trick works because the sender deliberately picks the checksum value that makes everything sum to all 1s. If the receiver's sum of data+checksum doesn't complement to all zeros, *something* changed in transit — but checksum, like simple parity, can't correct the error or guarantee detection of every possible corruption pattern; it's a lightweight, 'good enough for most cases' check, which is exactly why TCP/UDP use it (cheap to compute) while relying on retransmission, not correction, when it fails."

5. Cyclic Redundancy Check (CRC)

The strongest and most widely used detection scheme in real data-link protocols (Ethernet, etc.) — based on **binary (modulo-2) polynomial division**, where addition/subtraction is just XOR (no carries or borrows).

Sender-side procedure:

1. Agree on a **generator** (divisor) of **$r+1$** bits, representing a degree- **r** polynomial.
2. Append **r** zero bits to the end of the original data.
3. Divide this extended data by the generator using XOR-based binary division.
4. The **remainder** (exactly **r** bits) is the **CRC**.
5. Replace the appended zeros with this CRC to form the final transmitted codeword.

Worked example: data = **1101**, generator = **1011** (this is 4 bits, so **$r = 3$** — append 3 zeros).

```

Dividend: 1101000

1101000
1011
----
0110   (1101 XOR 1011)
  bring down next bit → 1100
1011
----
0111
  bring down next bit → 1110
1011
----
0101
  bring down next bit → 1010
1011
----
0001           ← remainder = 001 (last 3 bits)

```

CRC = 001. Transmitted codeword = data + CRC = **1101** + **001** = **1101001**.

At the receiver: divide the *received* codeword by the same generator.

```
1101001 ÷ 1011 → remainder = 000
```

A zero remainder means no error detected. (If the codeword had been corrupted in transit, dividing it by the generator would almost certainly produce a non-zero remainder, flagging the error — CRC's mathematical structure, based on the choice of generator polynomial, guarantees detection of all single-bit errors, all double-bit errors, all odd numbers of errors, and any burst error shorter than the generator's degree.)

Interview soundbite: "CRC is just polynomial division in disguise — the sender picks a CRC value that makes the transmitted codeword exactly divisible by the generator polynomial. The receiver redoes that same division; a non-zero remainder means the codeword is no longer cleanly divisible, which can only happen if something changed in transit. Its strength over parity/checksum comes from well-chosen generator polynomials that mathematically guarantee catching entire *classes* of common error patterns, not just a lucky subset."

6. Hamming Code — Error Correction, not just detection

Parity, 2D-parity, checksum, and CRC can all **detect** errors, but the standard fix on detection is to **ask for retransmission**. Hamming code goes further: it can **locate and correct** a single-bit error **without retransmission** at all.

The core idea

Instead of one parity bit covering *everything*, use **multiple parity bits**, each covering a carefully chosen, overlapping *subset* of positions — chosen so that the specific *combination* of which parity checks fail directly encodes the **binary position number** of the corrupted bit.

How many redundant bits (r) are needed? Must satisfy:

$$2^r \geq n + r + 1 \quad (n = \text{number of data bits})$$

Worked example: 4 data bits ($n = 4$) → try $r = 3$: $2^3 = 8 \geq 4 + 3 + 1 = 8$ ✓ (exactly satisfies it). Total codeword length = $n + r = 7$ (this is the classic **Hamming(7,4)** code).

Bit placement: parity bits go at positions that are **powers of 2** (1, 2, 4); data bits fill the rest (3, 5, 6, 7).

Position:	1	2	3	4	5	6	7
Bit:	p1	p2	d1	p3	d2	d3	d4

Which parity bit covers which positions? A position is covered by parity bit p_i if that position's binary representation has the corresponding bit set:

- p_1 (position 1) covers positions whose binary form has bit 0 set → **1, 3, 5, 7**

- **p2** (position 2) covers positions whose binary form has bit 1 set → **2, 3, 6, 7**
- **p3** (position 4) covers positions whose binary form has bit 2 set → **4, 5, 6, 7**

Worked example — encoding data 1101 (using odd parity): assign $d1=1$, $d2=1$, $d3=0$, $d4=1$ to positions 3, 5, 6, 7.

- **p1** covers $\{1,3,5,7\} = \{p1, 1, 1, 1\} \rightarrow$ data bits sum to 3 (odd) $\rightarrow$ set $p1 = 0$ to keep the group's total odd ($3+0=3$).
- **p2** covers $\{2,3,6,7\} = \{p2, 1, 0, 1\} \rightarrow$ data bits sum to 2 (even) $\rightarrow$ set $p2 = 1$ to make the group's total odd ($2+1=3$).
- **p3** covers $\{4,5,6,7\} = \{p3, 1, 0, 1\} \rightarrow$ data bits sum to 2 (even) $\rightarrow$ set $p3 = 1$ to make the group's total odd ($2+1=3$).

Final codeword (positions 1–7): **0 1 1 1 1 0 1**

Decoding — how the receiver locates a corrupted bit

The receiver recomputes each parity group's check. Each check that **fails** contributes its own position-value (1, 2, or 4) to a running sum; the resulting sum is the **exact position number of the corrupted bit**.

Worked example: suppose the codeword **0111101** is corrupted in transit to **0111111** (bit at position 6 flipped: **0** $\rightarrow$ **1**).

- Check **p1** group $\{1,3,5,7\} = 0,1,1,1 \rightarrow$ sum = 3 (odd) $\checkmark$ consistent with odd parity $\rightarrow$ **passes**.
- Check **p2** group $\{2,3,6,7\} = 1,1,1,1 \rightarrow$ sum = 4 (even) $\rightarrow$ **fails** (expected odd) $\rightarrow$ contributes **2**.
- Check **p3** group $\{4,5,6,7\} = 1,1,1,1 \rightarrow$ sum = 4 (even) $\rightarrow$ **fails** (expected odd) $\rightarrow$ contributes **4**.

Sum of failing groups' position-values = 2 + 4 = 6 $\rightarrow$ the corrupted bit is at **position 6** — exactly where we flipped it. The receiver simply flips position 6 back, **fully correcting the error with no retransmission needed**.

If all parity checks pass, the sum is 0 — no error. If the sum points to a position beyond the codeword length, or exactly one bit is inconsistent, that's how Hamming distinguishes "no error" from "single-bit error" — but note a classic limitation: **basic Hamming(7,4) can correct only one bit error per codeword; if two bits are corrupted, it will typically miscorrect (compute a wrong position) rather than flag the situation as uncorrectable**, unless extended with an additional overall parity bit (the common "SECDED" — Single Error Correction, Double Error Detection — extension).

Summary Comparison

Method	Can detect	Can correct	Overhead	Used at
Single parity	Odd number of bit flips only	No	1 bit	Simple links
2D parity	Most error patterns; pinpoints location	Effectively yes, for single-bit case	~1 bit per row + column	Simple links
Checksum	Most errors (not guaranteed for all patterns)	No	Fixed-size field (e.g., 16 bits)	Transport layer (TCP/UDP), IP header
CRC	Guaranteed classes: all single/double-bit errors, all odd-count errors, bursts shorter than generator degree	No	r bits (generator degree)	Data Link layer (Ethernet, etc.)
Hamming code	Single-bit errors (reliably located)	Yes, single-bit	r bits, $2^r \geq n+r+1$	Memory systems (ECC RAM), some link-layer contexts

Interview Questions With Answers

Q1. Why can't single parity check detect a 2-bit error?

Answer: Single parity only checks whether the *total count* of 1s in the unit matches the chosen convention (even or odd). Flipping any 2 bits either turns two 0s into two 1s, two 1s into two 0s, or one of each — in every case, the total count of 1s changes by an even number (net effect: 0, unless one flip cancels the other, in which case the parity string is unchanged), so the parity bit still matches its expected value and the error passes through completely undetected. Single parity can only reliably detect an *odd* number of bit flips.

Q2. How does 2D parity improve on single parity, and what's its remaining blind spot?

Answer: By computing a parity bit for both every row and every column, a single-bit error fails both its row's check and its column's check simultaneously, which not only detects the error but pinpoints its exact position (row/column intersection). Its remaining blind spot is a very specific 4-bit corruption pattern: if exactly 2 rows and 2 columns each have exactly one bit flipped, arranged so

each affected row and column still shows an even count of flips, the errors cancel out and go undetected — an unlikely but structurally real gap.

Q3. Why does checksum use one's complement addition specifically, rather than ordinary binary addition?

Answer: One's complement addition (where any carry-out from the most significant bit wraps around and is added back into the result) is specifically designed so that the sender can choose a checksum value making the total sum of data + checksum come out to all 1s (or equivalently, all 0s after complementing). This gives a clean, simple pass/fail test at the receiver: sum everything received (including the checksum) — if it complements to all zeros, nothing detectably changed; any bit flip in transit will, with high likelihood, break this exact arithmetic identity.

Q4. Explain, at a conceptual level, why CRC's polynomial-division approach can guarantee detection of certain error classes that checksum can't guarantee.

Answer: CRC treats the data as coefficients of a binary polynomial and picks a CRC remainder specifically so the transmitted codeword is exactly divisible (with zero remainder) by a carefully chosen generator polynomial. The mathematical properties of a well-chosen generator polynomial (e.g., one that isn't a factor of certain low-degree error patterns) guarantee that specific classes of corruption — every single-bit error, every double-bit error, every odd number of bit errors, and any burst error shorter than the generator's degree — will always change the codeword in a way that makes it no longer exactly divisible by the generator, producing a detectably non-zero remainder. Checksum's plain arithmetic sum doesn't have this same guaranteed structural property against those specific error classes — it can miss some corruption patterns that happen to still sum correctly.

Q5. What fundamentally distinguishes Hamming code from parity/checksum/CRC?

Answer: Parity, checksum, and CRC are all *detection-only* schemes — on failure, the receiver's only recourse is to discard the data and request retransmission. Hamming code uses multiple overlapping parity groups specifically constructed so that the *pattern* of which groups fail directly encodes the *position* of the corrupted bit, allowing the receiver to locate and flip it back — correcting the error without needing any retransmission at all.

Q6. In Hamming code, why are parity bits placed specifically at power-of-2 positions (1, 2, 4, 8, ...)?

Answer: Placing parity bits at power-of-2 positions means each parity bit's position, in binary, has exactly one bit set (e.g., position 4 = **100**) — which makes it naturally correspond to one specific bit of the binary "failure sum" the receiver computes during decoding. This design ensures that the sum of the position-values of all failing parity groups reconstructs the *exact* binary position number of the corrupted bit, rather than requiring any separate lookup or additional computation.

Q7. What is the key limitation of basic Hamming(7,4), and how is it addressed?

Answer: Basic Hamming(7,4) can reliably correct only a single-bit error per codeword; if two bits are corrupted simultaneously, the failing-parity-group calculation will typically point to some *other* position entirely (a miscorrection) rather than flagging the block as uncorrectable, silently producing wrong data. This is addressed by adding one additional overall parity bit covering the entire codeword (the SECDED extension — Single Error Correction, Double Error Detection), which lets the receiver distinguish "one error, correctable" from "two errors, must be flagged/retransmitted" rather than confidently miscorrecting.

Q8. Scenario: A noisy link experiences frequent 3-consecutive-bit burst errors.

Which of the methods covered here would you choose, and why?

Answer: CRC, with a generator polynomial of degree ≥ 3 . CRC's mathematical guarantee explicitly covers detection of any burst error shorter than the generator polynomial's degree, so a degree-3-or-higher generator is specifically guaranteed to catch every 3-bit burst. Single/2D parity offer no such structural guarantee against burst errors specifically (a burst could, in principle, flip an even number of bits within a single row in a way that a row-parity check would miss), and while Hamming code could in principle be extended, its natural strength is single-bit correction, not burst-error detection — CRC is the tool actually designed and mathematically proven for exactly this burst-error scenario.